# ML / DL Project Canvases

Eleven templates for getting a bird's-eye view of a machine-learning project — one canvas per layer of reasoning.

---

### How to use this deck

- Each canvas is a fixed template you fill in for any project. They are tools for thinking, not bureaucracy.
- Project Card, Shape Diagram, and Experiment Log are the three to revisit constantly. Start there.
- Add the others when you hit a problem they would have prevented. Don't try to maintain all eleven at once.
- The canvases are not strictly hierarchical — experts move between layers fluidly; novices try to climb a ladder.

### Index

# 01. Project Card

*Top-level map of the project on one page. If you can't fill it in, you don't yet have a project — you have code.*

**WHEN TO USE**

At the start of any project, and whenever you onboard someone (including future-you) to it. Pin it to the README.

**HOW TO FILL IT**

- Problem in one sentence — if it doesn't fit, you don't understand the problem yet.
- Input and output schemas: shape, dtype, range, meaning. Be exact.
- Loss vs. metric: these are different things. State both.
- Headline result with a confidence interval, against a stated baseline.
- Known failure modes — every project has them; pretending otherwise is the bug.

---

## Canvas 1 — Project Card

*Top-level map. One page. If you can't fill it, you don't have a project yet.*

**PROBLEM (one sentence)**
e.g. Classify chest X-rays into 5 pathologies for radiologist triage.

**INPUT SCHEMA**
shape · dtype · range · source

**OUTPUT SCHEMA**
shape · dtype · meaning

**DATASET**
name · size · splits · key biases

**MODEL FAMILY**
architecture · # params · pretrained?

**LOSS**
what objective is being minimized

**METRIC**
what we actually report · baseline value

**HEADLINE RESULT**
current best · CI · vs baseline

**KNOWN FAILURE MODES**
where the model breaks · why

**OPEN QUESTIONS / NEXT EXPERIMENTS**
the 2-3 things you'd run next if given a week

# 02. Math Card

*Three lines: the loss, the assumptions it encodes, and the update rule. Writing it down is the point.*

**WHEN TO USE**

When you adopt a loss or optimizer you haven't written down before. When debugging weird training behavior — the math card often reveals a violated assumption.

**HOW TO FILL IT**

- Write the loss in symbols, not in PyTorch. The framework hides what you need to see.
- List the assumptions baked in (i.i.d., calibrated probs, etc.). These are what break in deployment.
- Write the update rule including any clipping, normalization, or scheduling that modifies the raw gradient step.

---

## Canvas 2 — Math Card

*Three lines. Loss, assumption, update. Writing it down is the point.*

**1. LOSS FUNCTION**
Write the loss explicitly in math.

Example: $L(\theta) = -(1/N)\ \Sigma_i\ y_i \log p\_\theta(x_i) + \lambda\|\theta\|^2$

What does each term do? What does it penalize? What does it ignore?

**2. ASSUMPTIONS BEHIND IT**
What must be true about the data / world for this loss to be the right thing?

Example: samples i.i.d. from p(x,y); labels are clean;
classes are mutually exclusive; target is calibrated probabilities.

**3. UPDATE RULE**
How does θ change each step?

Example: $\theta \leftarrow \theta - \eta \cdot \hat{m}\_t / (\sqrt{\hat{v}\_t} + \varepsilon)$ (Adam)

Where do gradients come from? Anything stopped, clipped, or normalized?

# 03. Shape Diagram

*Boxes for modules, arrows annotated with tensor shapes and dtypes. Most bugs are visible here before you run anything.*

**WHEN TO USE**

Whenever you change architecture, swap a backbone, or integrate someone else's code. Re-draw, don't just patch.

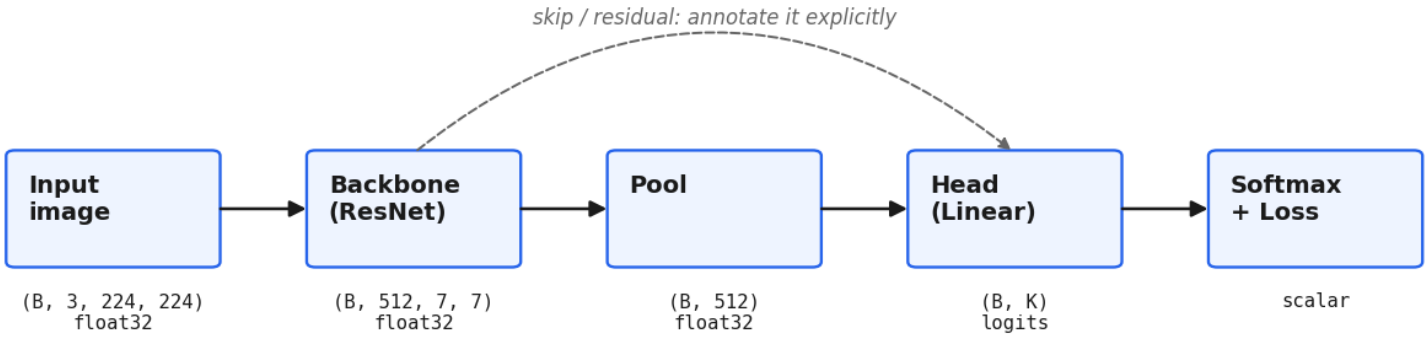
**HOW TO FILL IT**

- Every arrow gets a shape AND a dtype. Half-labeled diagrams are worse than none.
- Mark the batch dimension separately from feature dimensions.
- Draw all reshapes, permutes and squeezes — these are where bugs hide.
- Mark gradient barriers (.detach, stop_gradient, frozen modules) explicitly.

---

## Canvas 3 — Shape Diagram

*Boxes for modules. Arrows annotated with tensor shapes & dtypes. Mark where gradients stop.*




### GRADIENT FLOW

← ∂L/∂θ flows back through every solid arrow.
← Dashed arrows / .detach() / stop_gradient → mark with a barrier symbol ×.
← Frozen modules → shade them gray and note 'requires_grad=False'.

**CHECKLIST**

- ☐ every arrow labeled with shape AND dtype
- ☐ batch dim (B) marked separately from feature dims
- ☐ all reshapes / permutes / squeezes drawn (this is where bugs hide)
- ☐ device transfers (CPU→GPU) marked

# 04. Data Card

*Be specific about what you're training on. Adapted from HuggingFace and Google data cards.*

**WHEN TO USE**

Once per dataset, plus whenever you change preprocessing, augmentation, or split logic. The train-vs-deployment-gap row is the most important one.

**HOW TO FILL IT**

- Source and licensing — non-negotiable, especially for human data.
- Label process: who labeled it, how, and how much they agreed.
- Split logic: random, stratified, temporal, by-subject. Did you check for leakage?
- Train-vs-deployment gap: the single most useful row for predicting field failures.

---

## Canvas 4 — Data Card

*Where most bugs and most gains live. Be specific.*

**SOURCE**
where did it come from? license? collection date?

**SIZE**
# samples · # classes · raw storage size

**LABEL PROCESS**
human · weak · synthetic? · inter-annotator agreement?

**CLASS / TARGET DISTRIBUTION**
imbalance ratio · long-tail? · sketch the histogram

**SPLIT LOGIC**
random · stratified · temporal · by-subject? · leakage checked?

**PREPROCESSING**
resize · normalize · tokenize — applied where (CPU/GPU, train/eval)?

**AUGMENTATIONS**
what · probability · train-only? · semantics-preserving?

**KNOWN BIASES**
demographic, geographic, temporal skew

**TRAIN vs DEPLOYMENT GAP**
the single most important row. how do they differ?

# 05. Training Dashboard

*Not a static doc — a live view. The discipline is deciding in advance which curves you'll watch.*

**WHEN TO USE**

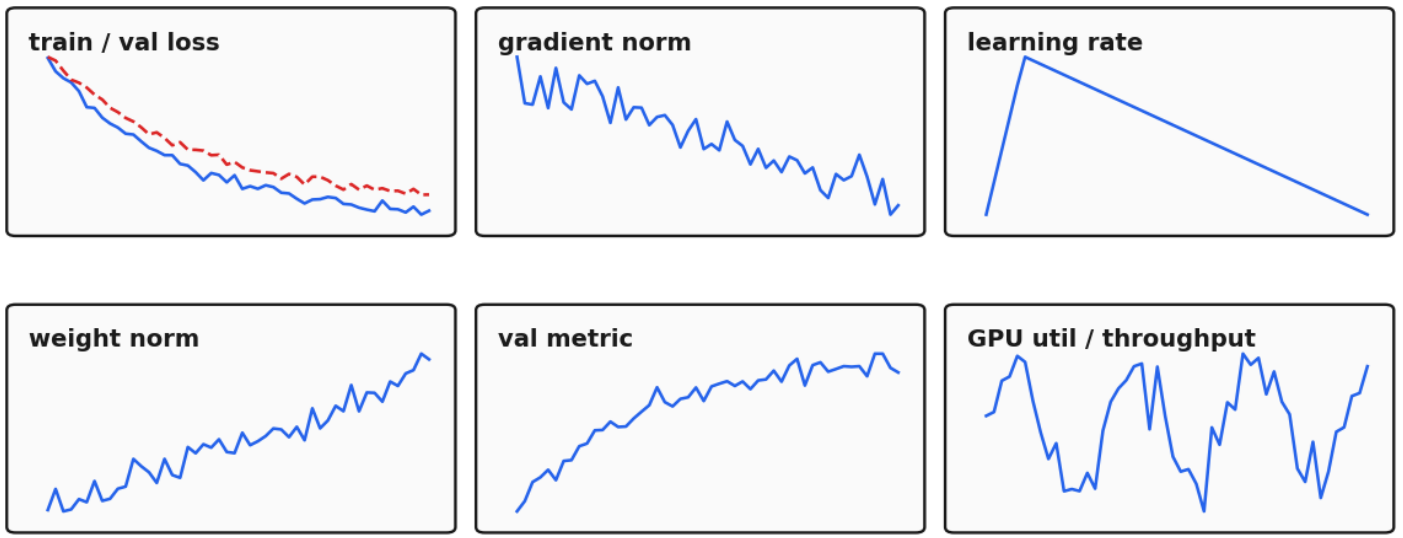
Every training run. Configure once per project; reuse across runs.

**HOW TO FILL IT**

- Pick the 6–8 panels BEFORE launching. Improvising panels mid-run leads to confirmation bias.
- Always overlay a baseline run. A single curve in isolation tells you nothing.
- Track gradient norm and weight norm — divergence usually shows up here before the loss curve does.
- Include a sample-predictions panel; numbers lie in ways that example outputs don't.

## Canvas 5 — Training Dashboard

*Decide IN ADVANCE which curves you'll watch. Live view, not a static doc.*




*+ sample predictions panel · + alert thresholds · + current run vs baseline overlay*

**RULES**

☐ pick panels before launching the run ☐ log to W&B / TensorBoard, not stdout ☐ overlay a baseline run, always

# 06. Repo Map

*The 5–7 files that matter, with one line each. New repos collapse to this skeleton.*

**WHEN TO USE**

When opening any new repo (yours or someone else's). Also when your own repo has grown beyond this — ask which files actually justify their existence.

**HOW TO FILL IT**

- Config is read first, always. If hyperparameters are scattered across .py files, fix that before anything else.
- train.py owns the loop, the optimizer, and the checkpointing — nothing else.
- eval.py should have no training side effects. Ever.
- notebooks/ exists for inspection only — never the source of truth for a result.

---

## Canvas 6 — Repo Map

*The 5-7 files that matter, with one line each. New repos collapse to this skeleton.*

| | |
|---|---|
| **project/** | |
| ├── config.yaml | all hyperparameters live here. nothing hardcoded in .py files. |
| ├── data.py | Dataset, DataLoader, augmentations. inputs → (x, y) batches. |
| ├── model.py | nn.Module(s). architecture only. forward() takes x → logits. |
| ├── train.py | training loop. optimizer, scheduler, checkpoint, logging. |
| ├── eval.py | metric computation on val/test. no training side effects. |
| ├── infer.py | load checkpoint → predict on new inputs. used in production. |
| ├── utils.py | seeds, device, logging helpers. nothing project-specific. |
| ├── **experiments/** | configs for each named experiment. one yaml per run. |
| ├── **notebooks/** | EDA & inspection only. never the source of truth for results. |
| ├── **tests/** | smoke tests: 1-batch forward pass, shape checks, eval determinism. |
| └── README.md | the Project Card (Canvas 1). first thing anyone reads. |

**WHEN OPENING A NEW REPO**
1. find these 7 files (or their equivalents). 2. read config first. 3. read data.py next. 4. then model, then train.
If the repo doesn't collapse to something like this → that itself is information.

# 07. Resource Profile

*A small table of where the project actually spends compute and memory. Tells you what to optimize.*

**WHEN TO USE**

Once per significant architecture or batch-size change. Before deciding to scale up — knowing your bottleneck saves more than guessing.

**HOW TO FILL IT**

- Measure, don't estimate. PyTorch profilers and nvidia-smi exist for this.
- Distinguish parameters, activations, and optimizer state — different memory footprints.
- Identify the single binding bottleneck (data / compute / memory / comms). That's what optimization should target.
- Re-measure after each significant change; bottlenecks shift.

---

## Canvas 7 — Resource Profile

*Fill in once per significant change. Tells you what to actually optimize.*

| COMPONENT | VALUE | UNIT | NOTES |
|---|---|---|---|
| model parameters | | M | trainable / total |
| activations memory (per sample) | | MB | at batch=1, fwd only |
| optimizer state | | × params | Adam=2x, SGD=0 or 1x |
| peak GPU memory (train) | | GB | at chosen batch size |
| training throughput | | samples/sec | steady state |
| inference latency (p50/p99) | | ms | batch=1, target device |
| data loader throughput | | samples/sec | without model |
| bottleneck | | — | data / compute / comms / memory |
| dtype strategy | | — | fp32 / amp / bf16 / fp16 |

**INTERPRETING THE BOTTLENECK**

data-bound → more workers, prefetch, lighter aug, faster format (webdataset, parquet)
compute-bound → mixed precision, fused ops, bigger batches, better kernels
memory-bound → grad checkpointing, ZeRO/FSDP, smaller batch, activation offload
comms-bound → reduce sync freq, overlap comms with compute, fewer GPUs sometimes faster

# 08. Eval Card

*What metric, what split, what baseline, what's the CI. If you can't say what a 1-point gain means, you don't have an eval — you have a number.*

**WHEN TO USE**

When defining a new project's success criteria. When a result feels surprising — usually the eval is wrong before the model is.

**HOW TO FILL IT**

- Declare ONE primary metric. Tracking many is fine; tying yourself to many lets you switch post-hoc.
- Compute a confidence interval. If it's wider than the gains you're celebrating, you're celebrating noise.
- Pre-commit to a minimum meaningful difference. Biggest single defense against fooling yourself.
- Look at sliced metrics, not just averages. Per-class, per-subgroup, per-difficulty.

---

## Canvas 8 — Eval Card

*What metric, what split, what baseline, what's the CI. If you can't say what a 1-point gain means, you don't have an eval.*

**PRIMARY METRIC**
the ONE number you optimize. why this one?

**SECONDARY METRICS**
what else you track. when do they override the primary?

**EVAL SPLIT**
size · how constructed · how it differs from train · last refreshed when?

**BASELINE(S)**
trivial baseline · prior SOTA · last released model

**CONFIDENCE INTERVAL**
how computed (bootstrap? multiple seeds?) · is the CI smaller than your gains?

**MIN MEANINGFUL DIFFERENCE**
below this Δ, treat results as equal. decide BEFORE running.

**SLICED METRICS**
per-class · per-subgroup · per-difficulty. where does the avg lie about the worst case?

**LEAKAGE CHECKS**
test seen during training? near-duplicates? overlapping users/time?

**FAILURE ANALYSIS**
look at N worst predictions. patterns? actionable?

# 09. Experiment Log

*One row per run. The 'conclusion' column is the one most people skip — it's the one that matters.*

**WHEN TO USE**

Continuously, for the life of the project. The log accumulates institutional memory; without it you re-run experiments you already ran.

**HOW TO FILL IT**

- Write the hypothesis BEFORE the run. Otherwise it becomes a story you tell about the result.
- Result must include a delta vs your declared baseline AND an uncertainty estimate.
- Conclusion is a sentence in plain English. 'kept' or 'rejected — too expensive' beats a number.
- Keep negative results. That is the entire point of the log.

---

## Canvas 9 — Experiment Log

*One row per run. The 'conclusion' column is the one people skip — it's the one that matters.*

| # | DATE | HYPOTHESIS | CHANGE VS BASELINE | Δ METRIC | CONCLUSION | NEXT |
|---|---|---|---|---|---|---|
| 001 | Mar 03 | baseline ResNet50 ok | — | 0.812 ± .004 | lock baseline | try aug |
| 002 | Mar 04 | RandAugment helps | + RandAug N=2 M=9 | +0.7 p=.02 | kept | tune M |
| 003 | Mar 05 | bigger > more aug | ResNet101, no aug | +0.2 n.s. | rejected — 2× cost | stop |
| 004 | Mar 06 | label smoothing helps | + LS=0.1 | +0.4 p=.04 | kept | combine |
| 005 | Mar 07 | combo > parts | RandAug + LS | +0.9 vs base | kept | scale data |
| ... | | | | | | |

**RULES OF THE LOG**

- ☐ one row per RUN, not per file edit.
- ☐ 'result' must include a Δ vs your declared baseline and an uncertainty estimate.
- ☐ 'conclusion' is a sentence in plain English. not a number.
- ☐ 'next' is concrete or empty. 'try more stuff' doesn't count.
- ☐ negative results stay in the log. that's the entire point.
- ☐ hypothesis written BEFORE the run.

# 10. Service Diagram

*Production view. Annotate every arrow with latency and throughput. Add the monitoring box and the failure paths.*

**WHEN TO USE**

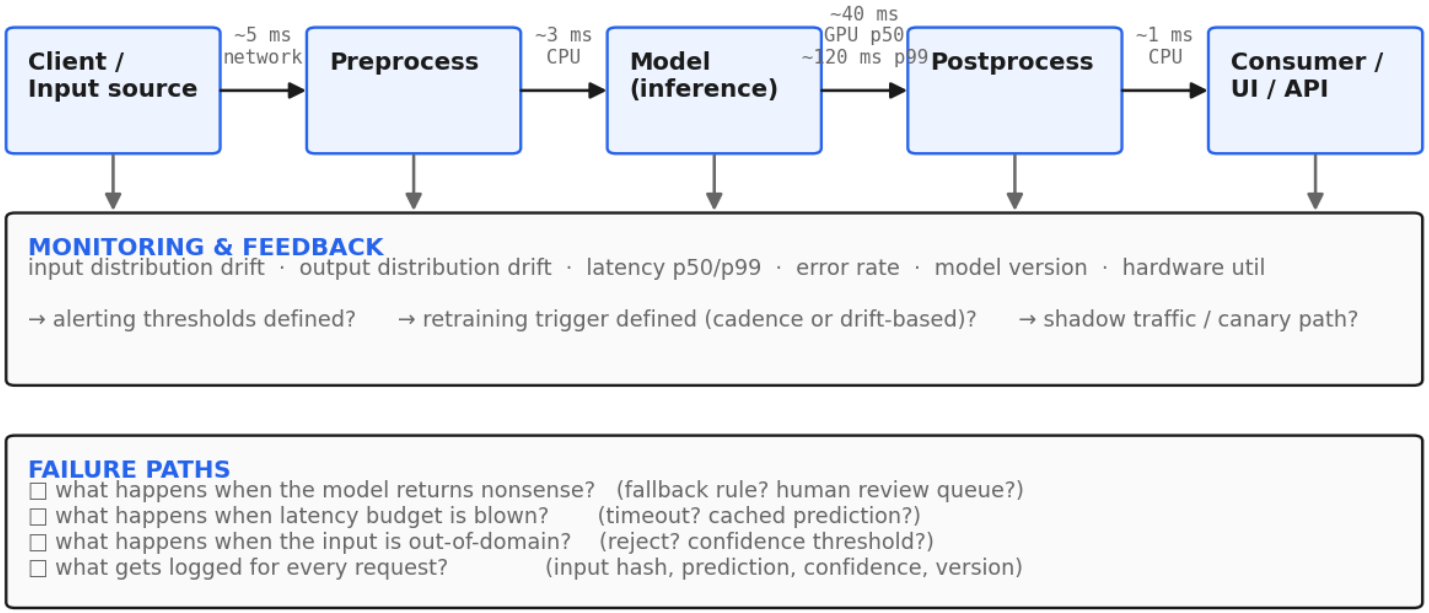
Before shipping. When investigating production incidents. When estimating cost-to-serve.

**HOW TO FILL IT**

- Annotate arrows with p50 AND p99 latency, not just average. Tails dominate user experience.
- Every node also feeds into a monitoring layer — input drift, output drift, errors, latency, version.
- Write down explicit failure paths: what happens when the model is wrong, slow, or out-of-domain?
- Define retraining triggers (cadence or drift-based) before the model goes live.

## Canvas 10 — Service Diagram

*Production view. Annotate every arrow with latency & throughput. Add the monitoring box.*



# 11. Risk Register

*Boring document. Worth doing.*

**WHEN TO USE**

At project start, and at every model version change. Especially important when the system affects people who can't easily push back.

**HOW TO FILL IT**

- Write risks in concrete user-affecting language, not 'accuracy degrades'.
- Likelihood × severity = priority. Spend mitigation effort proportionally.
- An unmitigated risk is itself a decision — record who made it.
- Red-team the list with someone outside the project. You will miss things alone.

## Canvas 11 — Risk Register

*Boring document. Worth doing.*

| # | FAILURE MODE | WHO IS AFFECTED | LIKELIHOOD | SEVERITY | MITIGATION |
|---|---|---|---|---|---|
| 01 | model under-predicts rare class | patients with X | med | high | class-balanced loss + per-class threshold |
| 02 | training data biased by site | out-of-site users | high | med | stratified eval + site-conditional finetune |
| 03 | drift after deployment | all users | high | med | monitor input stats + retrain cadence |
| 04 | PII leakage via memorized output | data subjects | low | high | deduplication + canary tokens + audit |
| 05 | adversarial inputs | platform integrity | low | high | input validation + rate limiting |
| 06 | | | | | |
| 07 | | | | | |
| 08 | | | | | |
| 09 | | | | | |

**HOW TO USE IT**

☐ write the risk in concrete user-affecting language. NOT 'model accuracy degrades'.
YES 'patients in age group X get false-negative diagnoses'.
☐ likelihood × severity = priority. spend mitigation effort proportionally.
☐ revisit at every model version. risks change with the data and the deployment.
☐ if a risk has no mitigation, that is itself a decision — record who decided to accept it.
☐ red-team this list with someone outside the project. you will miss things alone.